

MEMORY MANAGEMENT

VIRTUAL MEMORY, PAGING AND SEGMENTATION

Consolidated university study notes, comparisons, diagrams, formulas, and question bank

These notes consolidate the supplied material on logical and physical address spaces, memory partitioning, non-contiguous allocation and paging, virtual memory and demand paging, page faults and thrashing, and segmentation. Definitions, notation, examples, diagrams, and distinctions follow the source terminology.

Contents

1	Memory Management Fundamentals	3
1.1	Need for memory management	3
1.2	Logical and physical address spaces	3
1.3	Address binding	3
1.4	Dynamic relocation and protection	3
2	Contiguous Memory Allocation	4
2.1	Bare machine and resident monitor	4
2.2	Fixed partitioning	4
2.3	Variable partitioning	4
2.4	Placement strategies	4
3	Non-Contiguous Allocation and Paging	4
3.1	Concept of paging	4
3.2	Logical address format	5
3.3	Page-table translation	5
3.4	Worked paging examples	5
3.5	Paging advantages and limitation	5
4	Virtual Memory	5
4.1	Definition and need	5
4.2	Benefits and costs	6
5	Demand Paging and Page Faults	6
5.1	Demand paging operation	6
5.2	Page-fault sequence	6
5.3	Effective access time	6
5.4	Page replacement	6
5.5	Copy-on-write	7
6	Thrashing	7
6.1	Meaning and cause	7
6.2	Working set and control	7
7	Segmentation	7
7.1	User view of a program	7
7.2	Segment-table translation	7
7.3	Worked segmentation example	8

7.4 Advantages	8
8 Paging and Segmentation: Comparison	8
9 Important Formulas and Numerical Procedure	8
9.1 Address formulas	8
9.2 Procedure for paging numericals	8
9.3 Procedure for segmentation numericals	8
10 10-Mark Answer Templates	9
10.1 Explain paging and logical-to-physical address translation [10 marks]	9
10.2 Explain virtual memory and demand paging, including page-fault handling [10 marks]	9
10.3 Explain contiguous memory allocation and fragmentation [10 marks]	9
10.4 Explain segmentation with address translation and advantages [10 marks]	9
10.5 Compare paging and segmentation [10 marks]	9
11 Question Bank	9
12 Last-Minute Revision Page	10

Memory Management Fundamentals

Need for memory management

To improve CPU utilization and response speed, an operating system keeps several processes in main memory at the same time. Because processes share memory, the memory manager must maintain a safe and efficient allocation of the available space. Its main responsibilities are:

- keeping track of occupied and free memory;
- allocating memory when a process needs it and deallocating it when the process terminates;
- protecting the memory allocated to one process from unauthorized access by another;
- supporting the chosen allocation strategy, whether contiguous or non-contiguous.

Logical and physical address spaces

A **physical address** identifies an actual byte or word location in RAM. The set of physical addresses used by a program after it is loaded is its physical address space. A **logical address**, also called a virtual address, is generated by the CPU while a program executes. It is the address used by the program to refer to instructions and data; it is not necessarily the actual RAM location.

CPU generates logical address $\longrightarrow$ MMU translates address $\longrightarrow$ RAM receives physical address

The **Memory-Management Unit (MMU)** is the hardware device that performs the run-time mapping from logical to physical addresses. The logical address space is the set of logical addresses generated by a program; the physical address space is the set of corresponding locations in main memory.

Logical address: generated by the CPU and used by the program. Physical address: actual location understood by memory hardware. The MMU performs the translation.

Address binding

Address binding is the mapping of addresses associated with a program to memory addresses. It can occur at three times:

1. **Compile time:** if the memory location is known in advance, absolute code is generated and logical and physical addresses are the same.
2. **Load time:** if the location is not known at compile time, relocatable code is generated and converted to absolute code when the program is loaded.
3. **Execution time:** if the process may move during execution, binding is delayed until run time. Relocatable code is converted by hardware, usually the MMU.

Dynamic relocation and protection

In a simple relocation scheme, a **relocation register** (also called a base register) contains the smallest physical address assigned to a process. The CPU generates a logical address (or displacement), and the hardware adds the relocation-register value to obtain the physical address:

$$\text{physical address} = \text{relocation-register value} + \text{logical address}.$$

A limit register can store the size of the process. An address is legal only when it lies within the process's limit; otherwise the hardware raises a protection fault. Relocation and limit checking therefore provide both flexibility and protection.

Contiguous Memory Allocation

Bare machine and resident monitor

A bare machine has no operating system. The user program can occupy the whole memory and runs in kernel mode, so protection is not required between processes. In the resident-monitor approach, a monitor remains in memory and supervises user programs. The monitor handles job sequencing, loading, control transfer, error handling, and I/O routines, but memory protection becomes necessary because user programs share the system.

LOW ADDRESS	[Operating system / resident monitor]	[User process]	HIGH ADDRESS
-------------	---------------------------------------	----------------	--------------

Fixed partitioning

In fixed partitioning, memory is divided into a fixed number of partitions when the system is configured. One process is loaded into one partition. Partitions may be equal-sized or unequal-sized. Equal partitions are simple, but a process larger than a partition cannot be loaded. Unequal partitions can reduce wasted space when jobs have different sizes, but require a rule for choosing a suitable partition.

Internal fragmentation occurs when a process is smaller than its allocated partition and the unused space inside that partition cannot be assigned to another process. The number of simultaneously resident processes is limited by the number of partitions.

Variable partitioning

In variable (dynamic) partitioning, partitions are created according to the size of incoming processes. A process receives a contiguous block large enough for its address space. This avoids much internal fragmentation but creates holes between allocated blocks as processes terminate.

Feature	Fixed partitioning	Variable partitioning
Partition size	Fixed before execution	Created for each process
Main waste	Internal fragmentation	External fragmentation
Flexibility	Lower	Higher
Compaction	Usually unnecessary	May be needed to combine holes

Placement strategies

When a hole must be selected for a process, common strategies are:

- **First fit:** allocate the first hole large enough; fast and generally effective.
- **Best fit:** allocate the smallest hole that is large enough; leaves small leftover holes and may create many unusable fragments.
- **Worst fit:** allocate the largest hole; leaves a large remainder, but may waste large blocks.

External fragmentation is free memory split into small non-contiguous holes. The total free space may be sufficient, but no single hole may be large enough. **Compaction** moves allocated blocks so that the holes combine into one large block; it is possible only when relocation is dynamic and is expensive because data must be moved.

Non-Contiguous Allocation and Paging

Concept of paging

Paging allows parts of one process to occupy non-contiguous physical addresses. Physical memory is divided into fixed-size blocks called **page frames**; logical memory is divided into equal-size blocks called **pages**. A process's pages can be loaded into any available frames, so the pages need not be adjacent. The source notes use a typical page size of 4 KB and note that systems commonly support page sizes between 4 KB and 8 KB.

Logical memory: [Page 0] [Page 1] [Page 2] [Page 3] Physical memory: [Frame 3] [Free] [Frame 0] [Frame 7] [Free] [Frame 2]

Logical address format

The CPU-generated logical address is divided into a page number p (high-order bits) and a page offset d (low-order bits). If the logical address space is 2^m and page size is 2^n bytes, then:

$$\text{offset bits} = n, \quad \text{page-number bits} = m - n.$$

The offset identifies the exact byte within the page. Since pages and frames have identical sizes, the offset does not change during translation.

Page-table translation

Each process has a page table maintained by the operating system. The page number is used as an index; the selected entry gives the frame number. The MMU replaces the page number with the frame number and attaches the unchanged offset:

$$\text{logical address } (p, d) \longrightarrow \text{page table}[p] = f \longrightarrow \text{physical address } (f, d).$$

Worked paging examples

Example 1. A system has a 16-bit logical address and a page size of 4 KB. Since $4 \text{ KB} = 2^{12}$ bytes, the offset is 12 bits and the page number is $16 - 12 = 4$ bits. There are $2^4 = 16$ pages.

Example 2. A system has 8 pages of 8 KB each and 16 frames. Since $8 \text{ KB} = 2^{13}$, the offset is 13 bits and the page number uses 3 bits. For a logical address, divide by 2^{13} to obtain the page number and use the remainder as offset. Look up the page's frame f and compute $f \times 2^{13} + d$.

For a paging numerical: identify page size, calculate offset bits, split the logical address into page number and offset, read the frame number from the page table, and form the physical address using the same offset.

Paging advantages and limitation

Paging removes the need for a process to occupy consecutive physical memory and avoids external fragmentation. It supports efficient allocation and virtual memory. Its costs include page-table storage, address-translation overhead, and possible internal fragmentation in the final page. A translation lookaside buffer (TLB) may be used in practice to cache recent page-table entries.

Virtual Memory

Definition and need

Virtual memory is a memory-management technique that allows processes larger than available physical memory to execute. It gives each process the illusion of a large, continuous memory space by using secondary storage (disk or SSD) as an extension of RAM. Only required portions are loaded; inactive or not-yet-needed pages remain on secondary storage.

If RAM is 4 GB and a program is 8 GB, the operating system can keep frequently used pages in RAM and leave the remaining pages on disk. When a missing page is referenced, it is brought into a free frame or replaces another page. Virtual memory can be implemented using demand paging or demand segmentation; demand paging is commonly used because it is easier to implement.

Benefits and costs

Virtual memory allows large programs to run, increases the number of processes that can coexist, and improves memory utilization. It also introduces page-fault overhead and depends on secondary-storage performance. Excessive movement of pages can make the system spend more time paging than executing.

Demand Paging and Page Faults

Demand paging operation

Demand paging loads a page only when it is needed. Pages never referenced are never loaded. It combines paging with swapping and requires a page table and secondary storage. A contiguous area on disk called the process's **swap space** stores the virtual address space. A modified page may be written back to swap space when removed from RAM.

Each page-table entry includes a **valid bit**. In the source terminology, a valid bit of 1 indicates that the page exists in the virtual address space and is currently in memory; a bit of 0 indicates that it is not currently available in memory (or that the reference is invalid, depending on the implementation's additional status information). The term **lazy swapping** is used because a page is not brought into memory until it is required.

Page-fault sequence

1. The CPU generates a virtual address; the MMU indexes the page table.
2. If the valid bit is 1, the frame number is obtained and the memory access proceeds as in ordinary paging.
3. If the bit is 0, the MMU raises a page-fault (missing-page) interrupt and transfers control to the operating system.
4. The page-fault routine checks whether the requested address is valid for the process. An invalid reference causes the process to be terminated with an error.
5. For a valid reference, the OS locates the page on swap space and obtains a free frame. If no frame is free, it selects a victim page using a replacement algorithm.
6. The OS reads the required page into the selected frame, updates the page table and valid bit, and restarts the interrupted instruction.

```
Reference page  $p \rightarrow$  Page table  $\rightarrow$  valid? YES: access frame NO: page-fault handler
                                 $\rightarrow$  disk read  $\rightarrow$  update table  $\rightarrow$  restart
```

Effective access time

If a normal memory access takes t_m , a page fault takes t_f , and the page-fault rate is p , a simplified effective access time is:

$$EAT = (1 - p)t_m + p(t_f + t_m).$$

Because t_f is much larger than t_m , even a small page-fault rate can significantly increase the effective access time.

Page replacement

When no free frame exists, the OS chooses a victim page. A good replacement algorithm aims to minimize page faults. Important policies include:

- **FIFO:** replace the page that has been in memory the longest. It is simple but can show Belady's anomaly, where adding frames increases page faults.
- **Optimal:** replace the page whose next use is farthest in the future. It gives the minimum possible faults for a known reference string but cannot be implemented exactly online; it is a benchmark.
- **LRU:** replace the page that has not been used for the longest time. It uses the recent past as an approximation of the future and is usually better than FIFO, but tracking use can be costly.

Copy-on-write

Copy-on-write allows parent and child processes to initially share pages after process creation. A page is copied only when one process attempts to modify it. This reduces unnecessary copying and is especially useful when the child immediately replaces its address space with a new program.

Thrashing

Meaning and cause

Thrashing occurs when a process does not have enough frames to hold its actively used pages. It generates page faults continuously, causing frequent page-in and page-out operations. The CPU spends most of its time servicing faults rather than executing instructions, so CPU utilization and throughput fall sharply.

Working set and control

The **working set** is the set of pages actively used by a process during a recent time window. If the total working-set demand exceeds available frames, the system is at risk of thrashing. Control measures include reducing the degree of multiprogramming, allocating more frames to affected processes, using working-set or page-fault-frequency policies, and improving locality of reference.

Do not equate a single page fault with thrashing. A page fault is one event; thrashing is a sustained condition of excessive paging and poor useful CPU execution.

Segmentation

User view of a program

A user naturally views a program as a collection of logical segments: the main program, procedures, functions, arrays, variables, and other meaningful units. Segments are variable in size, and a segment's size may change during execution. Each segment has a segment number and each element is identified by an offset from that segment's beginning.

Segmentation implements this user view. A logical address is written (s, d) , where s is the segment number and d is the offset. Unlike paging, the division into segment number and offset is specified by the programmer/compiler rather than imposed as equal-size blocks by hardware.

Segment-table translation

The operating system maintains a segment table. Each entry contains:

- **segment base**: starting physical address of the segment;
- **segment limit**: length of the segment.

The segment number indexes the table. The MMU compares the offset with the limit. If $d < \text{limit}$, it computes:

$$\text{physical address} = \text{base} + d.$$

If the offset is outside the limit, an invalid-address error is generated.

Logical address $(s, d) \rightarrow \text{segment table}[s]$ gives $(\text{base}, \text{limit}) \rightarrow \text{check } d < \text{limit} \rightarrow$
 physical address $\text{base} + d$

Worked segmentation example

Suppose segment 2 has base 2200 and limit 780. For (2, 247), $247 < 780$, so physical address = $2200 + 247 = 2447$. Suppose segment 4 has base 1650 and limit 400. For (4, 439), $439 > 400$, so the reference produces an invalid-address error.

Advantages

Because a segment contains one type of object, protection can be assigned by meaning: a procedure may be execute-only, while a character array may be read-only. Segmentation also supports sharing code or data between processes; for example, a shared library can be placed in a segment shared by several processes.

Paging and Segmentation: Comparison

Feature	Paging	Segmentation
Division	Fixed-size pages	Variable-size logical segments
Address	Page number + offset	Segment number + offset
Who defines units?	System/hardware	Programmer/compiler view
Table entry	Frame number (plus status bits)	Base and limit
Fragmentation	Possible internal fragmentation	External fragmentation possible
Protection	Page-level permissions	Natural per-segment protection
Sharing	Page-based sharing	Meaningful code/data sharing
Translation	Page number indexes page table	Segment number indexes segment table

In paging, the offset is unchanged because page and frame sizes are equal. In segmentation, the offset must first be checked against the segment limit before it is added to the base.

Important Formulas and Numerical Procedure

Address formulas

$$p = \left\lfloor \frac{\text{logical address}}{\text{page size}} \right\rfloor, \quad d = \text{logical address mod page size}.$$

$$\text{Physical address (paging)} = \text{frame number} \times \text{page size} + d.$$

$$\text{Physical address (segmentation)} = \text{segment base} + d, \quad \text{provided } d < \text{segment limit}.$$

Procedure for paging numericals

1. Convert KB or MB to a power of two in bytes.
2. Calculate the offset and page number.
3. Divide the logical address into page number and offset.
4. Use the page table to find the frame number.
5. Combine frame number and unchanged offset.

Procedure for segmentation numericals

1. Read the segment number and offset.
2. Locate the segment-table entry.
3. Compare offset with limit.
4. If legal, add base and offset; otherwise report an invalid-address error.

10-Mark Answer Templates

Explain paging and logical-to-physical address translation

[10 marks]

Define paging; explain pages and frames; draw the logical-memory/physical-memory relationship; divide the address into page number and offset; describe the page table and MMU translation; include the formula and a numerical example; state advantages and the limitation of page-table overhead/internal fragmentation.

Explain virtual memory and demand paging, including page-fault handling

[10 marks]

Define virtual memory and its need; explain RAM plus secondary storage; describe demand paging and swap space; explain the valid bit; write the page-fault sequence in order; mention frame allocation/page replacement; conclude with benefits and page-fault overhead.

Explain contiguous memory allocation and fragmentation

[10 marks]

Introduce memory allocation; describe bare machine/resident monitor; explain fixed and variable partitioning; distinguish internal and external fragmentation; explain first fit, best fit and worst fit; describe compaction and its condition; include a comparison table.

Explain segmentation with address translation and advantages

[10 marks]

Start with the programmer's logical view; define segments and (s, d) ; explain base and limit fields; show the limit check and physical-address formula; solve one valid and one invalid example; finish with protection and sharing advantages.

Compare paging and segmentation

[10 marks]

Define both schemes; compare unit size, address format, table contents, fragmentation, protection, sharing and translation; draw both address-translation flows; conclude when each view is useful.

Question Bank

Definitions and short questions

1. Define logical address and physical address.
2. What is the role of the MMU?
3. Define address binding.

4. Distinguish compile-time, load-time and execution-time binding.
5. What is a relocation register?
6. What is fixed partitioning?
7. Define internal fragmentation and external fragmentation.
8. What is compaction?
9. Define a page and a page frame.
10. What are the two fields of a logical address in paging?
11. What is the purpose of a page table?
12. Why is the page offset unchanged during translation?
13. Define virtual memory.
14. What is demand paging?
15. What is swap space?
16. What is lazy swapping?
17. What is a valid bit?
18. Define a page fault.
19. What is page replacement?
20. Define thrashing.
21. What is a working set?
22. Define segmentation.
23. What are the fields of a segment-table entry?
24. What condition produces an invalid-address error in segmentation?

5-mark questions

1. Explain the need for memory management in a multiprogrammed system.
2. Compare logical and physical address spaces.
3. Explain the three times at which address binding may occur.
4. Explain dynamic relocation using a base/relocation register.
5. Describe fixed partitioning and its limitations.
6. Describe variable partitioning and external fragmentation.
7. Compare first fit, best fit and worst fit.
8. Explain paging with a suitable diagram.
9. Derive the page-number and offset bit counts for a 2^m address space and 2^n page size.
10. Explain page-table-based address translation.
11. Explain why virtual memory is needed.
12. List and explain the steps in handling a page fault.
13. Compare FIFO, optimal and LRU page replacement.
14. Explain thrashing and two methods to control it.
15. Explain segmentation and the role of base and limit.
16. State the advantages of segmentation for protection and sharing.

10-mark questions

1. Explain memory management, address spaces, address binding and MMU-based relocation.
2. Explain contiguous allocation, fixed and variable partitions, placement strategies and fragmentation.
3. Explain paging, page tables and logical-to-physical address translation with a numerical example.
4. Explain virtual memory and demand paging in detail.
5. Explain page faults, page replacement and effective access time.
6. Explain thrashing, working set and methods for reducing thrashing.
7. Explain segmentation, translation, protection and sharing with examples.
8. Compare paging and segmentation in a structured table.

Last-Minute Revision Page

CORE IDEA The OS maps program addresses to safe physical memory and allocates memory among processes.

ADDRESS SPACES Logical/virtual = CPU-generated. Physical = actual RAM location. MMU translates.

BINDING Compile, load, or execution time. Execution-time binding supports movement and uses hardware.

CONTIGUOUS Fixed partitions cause internal fragmentation. Variable partitions cause external fragmentation. Compaction combines holes.

PAGING Logical address (p, d) ; page table maps page p to frame f ; physical address (f, d) .

VIRTUAL MEMORY Only needed pages are in RAM; the rest remain in swap space.

PAGE FAULT Invalid bit/reference -> interrupt -> validate -> find frame/victim -> disk read -> update

table -> restart.

REPLACEMENT FIFO oldest; optimal farthest future; LRU least recently used.

THRASHING Too few frames for active pages -> repeated faults -> low useful CPU work.

SEGMENTATION Logical address (s, d) ; table gives base and limit; check $d < limit$; address = $base + d$.

DO NOT CONFUSE Page/frame are fixed-size. Segment sizes vary. Page offset is not remapped; segment offset is checked. One fault is not thrashing.

DIAGRAMS TO DRAW MMU translation, partitioned memory, pages-to-frames mapping, page-fault flow, segment-table translation.

NUMERICAL CHECK Paging: divide by page size. Segmentation: compare offset with limit before adding base.

Source Scope

This document was prepared from the supplied PDFs: *Logical and Physical Address Space–Memory Partitioning*, *Non-contiguous Memory Management–Paging*, *Virtual Memory–Demand Paging–Page Fault–Thrashing*, and *Segmentation*, using the supplied LaTeX notes template as the structural and visual baseline. The document preserves source terminology and includes practical diagrams, tables, formulas, examples, comparisons, and examination question banks.

Study principle: understand the address-translation mechanism first, then practise the numerical procedure and answer structure.